Network Science Lab, Week 4

Objective: By the end of this worksheet, you will be able to:

- Install Graphviz and its Python interface.
- Create and render graphs.
- Perform basic operations like adding nodes, edges, and applying layouts.

Deliverables: You don't need to send these to me. Just make sure you save them in a folder.

Part 1: Installation

A. Install Graphviz

- **Linux (Ubuntu/Debian):**

```
sudo apt-get install graphviz
```

- **macOS (Homebrew):**

```
brew install graphviz
```

- **Windows:**

  Download the installer from: https://graphviz.org/download/

  Install and add Graphviz's bin/ folder to your **PATH**.

B. Install Python Library graphviz

Part 2: Creating Your First Graph

Now that Graphviz is installed, it's time to create your very first graph. Instead of giving you the full code, here are the **most useful functions** you'll need. Your task is to figure out how to combine them into a working program that builds and renders a simple graph.

Core Functions in the `graphviz` Python package

1. **Graph Objects**
   - `graphviz.Digraph(comment="...")` → Create a **directed graph**.
   - `graphviz.Graph(comment="...")` → Create an **undirected graph**.
2. **Adding Nodes**
   - `.node(name, label=None, **attributes)`
     - `name`: unique ID (string).
     - `label`: what shows up in the graph.
     - `attributes`: shape, color, style, etc.
3. **Adding Edges**
   - `.edge(tail_name, head_name, **attributes)` → Add a single edge.
   - `.edges(list_of_strings)` → Add multiple edges quickly (e.g., `['AB', 'BC']`).
4. **Rendering and Viewing**
   - `.render(filename, format="png", view=True)` → Save and open the graph.
   - `.source` → Print the raw Graphviz source code for inspection.
5. **Customizing Layout Engine**
   - `.engine = "dot" | "neato" | "circo" | "fdp"`

Using the functions above to create a **directed graph** with the following requirements:

- 3 nodes:
  - Node A → labeled "Start"
  - Node B → labeled "Process"
  - Node C → labeled "End"
- Connect them with arrows: A → B → C
- Render the graph as a PNG file and make sure it opens automatically.
- Print the `.source` to see the underlying DOT language.

Make It Your Own

Once you've built the basic graph, experiment with these modifications:

1. **Styling**
   - Give Node A a green, filled circle.
   - Make Node B a box with a lightblue background.
   - Make Node C a diamond shape.
2. **Layout**
   - Render the same graph with two different engines (`dot`, `circo`, or `neato`) and compare the layout differences.
3. **Experiment with Multiple Edges**
   - Add an extra edge from Node A directly to Node C.
   - Use `.edges()` to add multiple connections in one line.

4. **Self-Check**
    - What happens if you don't provide a `label`?
    - Try printing `.source` again — can you understand how your graph is translated into DOT language?

Deliverables 1:

- One PNG file of the **basic graph**.
- One PNG file of your **styled and customized version**.
- A copy of the `.source` output in your notebook/script.

Part 3: Basic Operations with Graphviz

Now that you've built your first simple graph, let's go further. Here are the **essential functions and attributes** you'll need. Your job is to **experiment, combine, and test** them to unlock more possibilities.

Useful Functions and Attributes

1. **Adding Many Nodes**
    - Use a Python `for` loop with `.node(name, label)` to quickly generate nodes.
2. **Adding Many Edges**
    - `.edges(['12', '23', '34'])` → Add multiple connections in one step.
    - `.edge('1', '2', label="yes")` → Add a labeled edge.
3. **Customizing Nodes**
    - Attributes:
        - `shape` → e.g., `"box"`, `"ellipse"`, `"diamond"`, `"circle"`.
        - `color` → outline color.
        - `style="filled"` + `fillcolor="..."` → background color.
        - `fontsize` → adjust text size.
4. **Customizing Edges**
    - Attributes:
        - `label` → add text on the edge.
        - `color` → change edge color.
        - `style="dashed"` or `"bold"`.
5. **Choosing Layouts**
    - `.engine = "dot"` (hierarchical, top–down).
    - `.engine = "neato"` (spring-model).
    - `.engine = "circo"` (circular).

Expand the Graph

- Create **5 nodes** labeled `Node 1` through `Node 5`.
- Connect them in a chain: `1 → 2 → 3 → 4 → 5`.
- Add an extra edge from `1 → 5`.
- Render it in at least **two different layouts** (`dot` and `circo`).

Make It Visual

- Pick one node and style it differently (shape + fill color).
- Style one edge with a label and dashed line.
- Increase the font size of the node labels.

Build a Mini Workflow

- Create a **data pipeline graph** with the following steps:
  - Collect Data → Clean Data → Train Model → Evaluate → Deploy
- Add an edge from `Evaluate` back to `Clean Data` (to represent iteration).
- Style the `Train Model` node as a **box** with a yellow background.

Challenge

- Experiment with **two different layouts** for your workflow graph.
- Print the `.source` for your styled workflow — can you identify how **node attributes** and **edge labels** appear in DOT format?

Deliverables 2:

- PNG images of:
  1. Expanded graph
  2. Styled graph
  3. Workflow graph
- Copy of `.source` for at least one graph.